
INTERNAL DEVELOPER PLATFORM · CAPSTONE

Platform Engineering Reference Architecture

The capstone IDP mapped onto the CNCF Platform Working Group's five-plane reference model — showing how each plane is realised by concrete, running components.

Developer Control

Integration & Delivery

Resource

Security

Monitoring & Logging

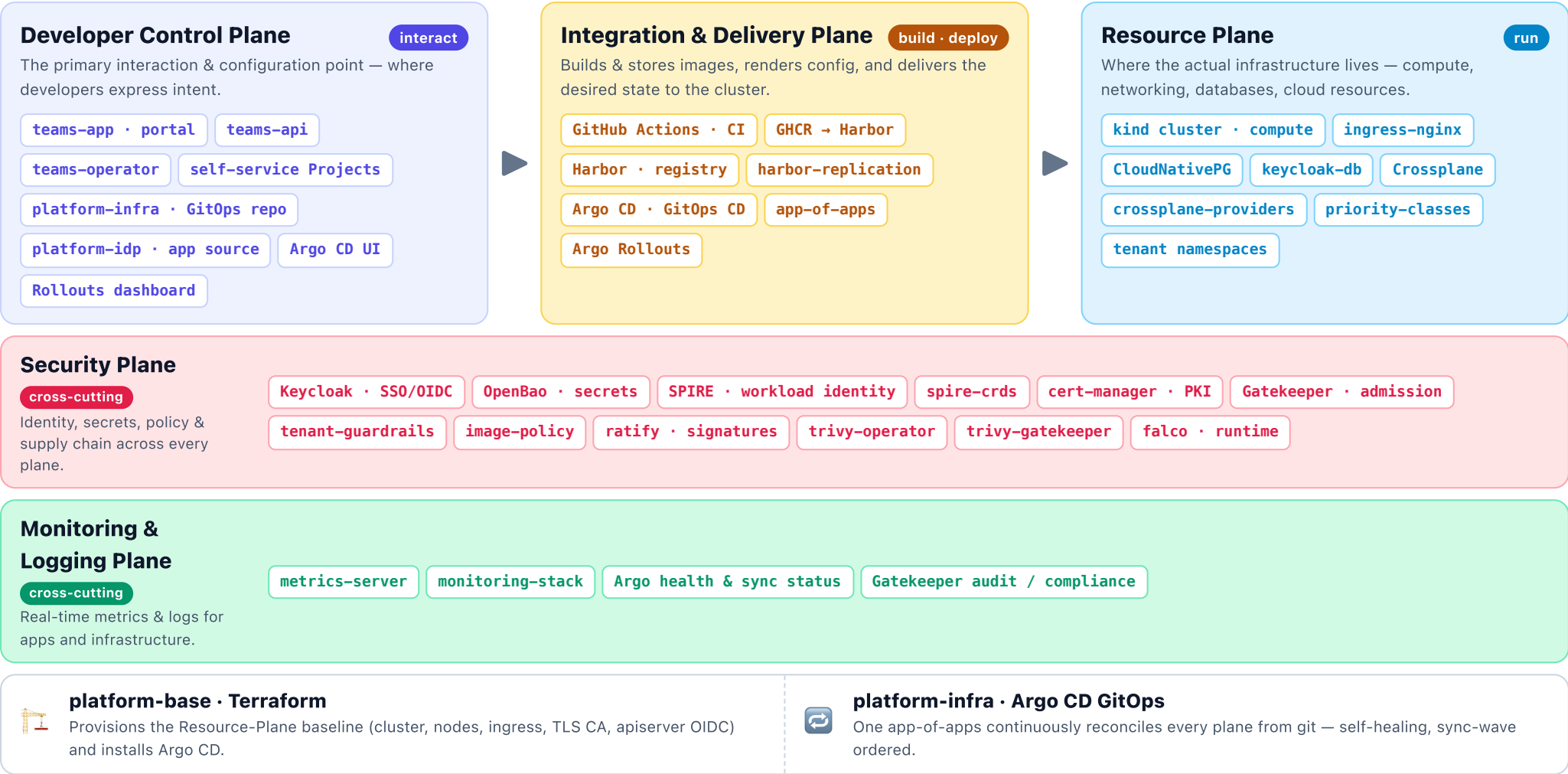
Model Five planes per *platformengineering.org* — Developer Control, Integration & Delivery, Resource (the flow from intent to running workload), with Security and Monitoring & Logging as cross-cutting planes.

Realisation A local `kind` cluster, provisioned by Terraform (`platform-base`) and delivered by Argo CD GitOps (`platform-infra`).

THE REFERENCE ARCHITECTURE

Five planes, one platform

DEVELOPER INTENT ————— RUNNING WORKLOAD



What each plane is — and how the capstone realises it

Plane	Purpose (reference model)	Capstone realisation
Developer Control	The primary configuration layer and interaction point for platform users; where intent is expressed.	The teams-* self-service IDP — <code>teams-app</code> portal, <code>teams-api</code> , and <code>teams-operator</code> reconciler — plus the GitOps config repo (<code>platform-infra</code>) and app source (<code>platform-idp</code>). Developers order Projects/namespaces and connect repos; they never touch cluster internals.
Integration & Delivery	Build and store the image, render app/infra config, and deploy the final state.	GitHub Actions builds images to GHCR , mirrored into Harbor (<code>harbor-replication</code>). Argo CD is the GitOps CD engine (app-of-apps); Argo Rollouts does progressive delivery.
Resource	Where the actual infrastructure exists — clusters, databases, storage, networking, DNS.	The kind cluster (compute), ingress-nginx (networking), CloudNativePG + <code>keycloak-db</code> (databases), Crossplane + providers (self-service cloud resources), priority classes, and the tenant namespaces where workloads run.
Security (cross-cutting)	Manage secrets and identity; protect sensitive information; enforce policy and supply-chain integrity.	Keycloak (SSO/OIDC), OpenBao (secrets), SPIRE (SPIFFE workload identity), cert-manager (PKI); Gatekeeper + <code>tenant-guardrails</code> (admission policy); image-policy/ratify (signature & attestation), trivy (CVE scanning), falco (runtime).
Monitoring & Logging (cross-cutting)	Real-time metrics and logs for applications and infrastructure.	metrics-server and the monitoring-stack ; supplemented by Argo CD health/sync status and Gatekeeper audit surfaced in the portal's compliance view.

Why the flow is Control → Delivery → Resource. A change starts as developer intent in the Control Plane (an order in the portal, a commit to a config repo), is built and reconciled by the Delivery Plane, and lands as running infrastructure in the Resource Plane — the classic left-to-right IDP value stream.

Why Security & Monitoring are cross-cutting. They are not a stage in the flow — every plane authenticates via Keycloak, pulls secrets from OpenBao, is gated by Gatekeeper, and is observed by the monitoring plane. They wrap the whole platform rather than sitting in the pipeline.